

Implementing OpenStreetMap for a Real-World Game World

Executive summary

OpenStreetMap is an excellent semantic base layer for a real-world game world because it encodes roads, buildings, POIs, water, land use, and many other ground features as tagged nodes, ways, and relations. It is not, by itself, a terrain source, a guaranteed survey-grade dataset, or a complete source of micro-detail. In practice, the most accurate results come from treating OSM as the authoritative **vector semantics** layer, then combining it with a DEM or LiDAR source for elevation and a procedural/art layer for missing detail.

1

For most production pipelines, the safest default is to keep OSM in `.osm.pbf` as the source of truth, preprocess it with **osmium/libosmium**, **osm2pgsql**, **GDAL/OGR**, and **PROJ**, and only convert to engine-native meshes, terrain tiles, navmeshes, colliders, and metadata at the end of the pipeline. PBF is the recommended OSM processing format because it is compressed and optimized for processing; GeoJSON and shapefiles are better treated as filtered interchange formats, not the master archive.

2

For acquisition, use the smallest possible source that fits your scope. The OSM Export tab is intentionally limited to small areas at high zoom. Public Overpass is best for targeted prototypes and analytical pulls, not repeated bulk production jobs. Regional extracts from Geofabrik or BBBike are usually the operational sweet spot for a city, state, or country. The weekly planet dump is the right entry point only if you need your own global mirror or a continuously updated backend.

3

On terrain, use **USGS 3DEP DEMs/LiDAR** wherever you can in the United States, because they are high-quality and public domain. For near-global coverage, **SRTM** is still the most practical baseline. **ASTER GDEM** is a useful fallback when coverage matters, especially at higher latitudes than SRTM, but it is not a substitute for good LiDAR where precision matters. DEM reprojection, mosaicking, clipping, resampling, and contour generation are mature GDAL tasks; point-cloud conditioning is where PDAL belongs.

4

For engine integration, the most durable recommendation is **not** to depend on a thin “OSM importer plugin” as the core of the production pipeline. Instead, build an offline GIS preprocessing stage and feed the engine native assets. Use **Cesium for Unity** or **Cesium for Unreal** when you truly need globe-scale georeferencing, high-accuracy WGS84 positioning, and streaming 3D Tiles. For city- or region-scale games, a local projected CRS plus chunked offline asset compilation is usually simpler, faster, and easier to validate. Unity needs origin shifting or a high-precision framework for very large worlds; Unreal has Large World Coordinates and World Partition; Godot has large-world coordinates, native chunked navmesh baking, and a capable ecosystem around Terrain3D and procedural meshes.

5

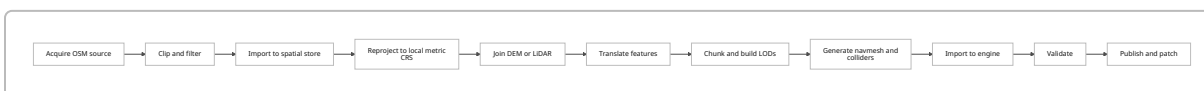
Licensing is manageable but must be designed in from the start. OSM data is under the ODbL; attribution must be visible to users of the produced work; and if you publicly distribute an **adapted database**, the share-alike rules may apply. The cleanest operational pattern is to maintain separate layers for raw/adapted

OSM-derived data, generated game assets, and non-OSM procedural/art content so that updates, validation, and legal review are not entangled. This report is technical guidance, not legal advice. 6

Decision context and recommended architecture

Several critical implementation details are unspecified: the geographic area of interest, required horizontal and vertical accuracy, online versus offline distribution, art style, whether the world should remain geodetically correct at full planetary scale, and the target platforms' memory/storage budgets. Because those choices dominate architecture, the recommendations below assume a platform-neutral workflow that must scale from a city-sized open world to a large regional map, while preserving a path to globe-scale georeferencing if needed.

The highest-confidence architecture is an **offline-first GIS pipeline** that ingests OSM vector data and elevation data into a preprocessing stack, reprojects everything into a metric working CRS, generates engine-native chunks, and only then hands off to runtime streaming systems. That approach aligns with the intended strengths of OSM PBF, osm2pgsql, GDAL/OGR, PROJ, and engine-native large-world systems. 7



In practical terms, the “source of truth” layer should stay as **PBF + replication metadata + raw DEM/LiDAR**, while the “working” layer should live in **PostGIS or equivalent file-based chunk stores**, and the runtime layer should be **engine-native assets plus lightweight gameplay metadata**. That division keeps reprojection, differential updates, and legal attribution easier to reason about than a plugin-centric pipeline that tries to do everything inside the engine editor. 8

A good default stack for a new project is this: **Geofabrik/planet extract → osmium/libosmium for clipping and tag filtering → osm2pgsql flex into PostgreSQL/PostGIS → GDAL/OGR + PROJ for conversion/reprojection/validation → custom chunk compiler → Unity/Unreal/Godot native runtime systems**. Use **OSMnx** and **GeoPandas** as the fast Python prototyping layer around that core, especially for road graphs and analyst-facing QA notebooks. 9

Data acquisition, formats, and core tools

Data acquisition strategy

OSM supports multiple acquisition paths, but they serve different jobs. The website Export tab intentionally works only at high zoom for small areas. The editing API is explicitly for editing, not read-only or bulk extraction. Public Overpass instances are shared infrastructure that defend themselves against overuse, and the Overpass manual explicitly recommends that heavy users set up their own instance. 10

For production workflows, the most stable external source is usually **Geofabrik**, which publishes free OSM extracts updated daily and also offers raw formats and selected derived vector exports. **BBBike** is useful when you need a custom metro-scale area in one of many supported formats. When you need your own continuously updated global or continental backend, the **weekly planet dump** is the canonical source; as

of 2026-05-01, the current-state planet PBF was 86.1 GB compressed, and the XML equivalent was far larger. ¹¹

Overpass is best when you need **selective, query-driven extraction** rather than full coverage. It can return OSM XML, OSM JSON, derived GeoJSON-like outputs, and CSV-like tables. Its most common failure mode in game pipelines is asking `out geom` on relations and accidentally pulling geometry far outside the visible AOI; the manual explicitly documents output-side bbox cropping to constrain that. ¹²

A sensible scope-based rule is this. For a one-off prototype or city block, use Overpass or the site export. For a city, state, or country, start from a regional PBF extract. For a live service or very large world, use planet PBF plus replication metadata and keep the pipeline diff-aware from day one. ¹³

Data formats comparison

The most important design decision about formats is to distinguish **master ingest/storage formats** from **exchange/debug formats**. PBF wins for ingest; GeoJSON wins for convenience; shapefile is legacy-only unless you are integrating with older GIS tools. ¹⁴

Data format	Pros	Cons	Typical use cases	Performance notes
OSM XML	Human-readable; native API/export-oriented representation of OSM primitives. ¹⁵	Very large when decompressed; slower and more memory-hungry if handled naively; no official XSD schema. ¹⁵	Small exports, debugging, interoperability with older tools/plugins that expect <code>.osm</code> . ¹⁶	Prefer streaming/SAX-style parsing, not full DOM loading, for anything beyond small extracts. ¹⁵
OSM PBF	Highly compressed; recommended for data processing; supports replication metadata; faster to read/write than gzipped XML in the format notes. ¹⁷	Less inspectable by hand; requires OSM-aware tools. ¹⁸	Master source files, regional extracts, planet ingestion, diff-aware pipelines. ¹⁹	Best overall ingest format; block-based binary layout is designed for efficient decoding and future random access. ¹⁸

Data format	Pros	Cons	Typical use cases	Performance notes
GeoJSON	Ubiquitous in GIS/ web tooling; easy to inspect; convenient for engine-side debug importers and Python toolchains. ²⁰	RFC 7946 uses WGS84 geographic coordinates in decimal degrees; text-heavy; not a faithful one-to-one representation of raw OSM semantics unless explicitly derived. ²¹	Filtered interchange after clipping/ reprojection decisions are made, debug layers, editor tools, quick engine prototypes. ²²	Fine for small/ medium subsets; poor choice for planet-scale or highly repetitive batch processing. ²⁰
Shapefile	Broad legacy GIS compatibility. ²³	Attribute names limited to 10 characters; limited field types; sidecar-file complexity and truncation risks. ²³	Legacy interchange with existing GIS software or contractors who still require it. ²³	Acceptable for simple exchange, but not a good long-term internal schema for rich OSM tagging. ²³

Tools and libraries comparison

The tools below are the most useful “backbone” set for an OSM-to-game pipeline. The pattern is complementary, not exclusive: osmium for fast raw-file operations, osm2pgsql for structured/updatable storage, GDAL/OGR for format and CRS work, and OSMnx for rapid Python analysis and graph extraction. ²⁴

Tool / library	Pros	Cons	Typical use cases	Performance notes
osm2pgsql	Purpose-built OSM ETL into PostgreSQL/ PostGIS; flex output gives Lua-controlled schema mapping; supports updatable databases. ²⁵	Requires PostgreSQL/ PostGIS and either SQL or Lua fluency. ²⁶	Production spatial store, tiling database, update-capable backend, analytics/QA staging. ²⁷	<code>--slim</code> is needed for later updates and trades speed for lower RAM demand; <code>--flat-nodes</code> is recommended for planet/very large imports and stores node locations much more efficiently than PostgreSQL alone. ²⁸

Tool / library	Pros	Cons	Typical use cases	Performance notes
Osmium / libosmium	Fast clipping, conversion, sorting, tag filtering, diff handling; reference-complete extract strategies are well documented. ²⁹	Lower-level than a database ETL; some diff operations keep change content in memory. ³⁰	Extracting AOIs from large PBFs, applying changes, tags-based preprocessing, raw-file QA. ³¹	Excellent first-stage tool before database import; <code>apply-changes</code> memory use scales roughly with the in-memory change content. ³⁰
OSMnx	High-level Python interface for downloading road graphs and OSM features; convenient stats and simplification tooling. ³²	Not intended as a full planet ETL system. ³³	Prototype road-network extraction, graph validation, gameplay routing experiments, QA notebooks. ³⁴	Very productive for city/regional analysis; use it upstream of custom compilers rather than as final runtime data storage. ³⁵
GDAL / OGR	Mature universal conversion, clipping, reprojection, selection, and validity-repair tooling. ³⁶	Not OSM-semantic by itself; best as a geospatial utility layer, not your only OSM parser. ³⁷	Reprojection, layer export, clipping, contour generation, geometry repair, conversion to engine-friendly intermediate files. ³⁸	Strong batch performance and streaming-friendly CLI tools make it ideal for build automation. ³⁷
osmconvert	Fast OSM format conversion and processing; fewer dependencies than a full DB pipeline. ³⁹	Fewer functions than tools like Osmosis; no database access. ⁴⁰	Fast format conversion, clipping, pre-processing small to medium extracts. ⁴⁰	Commonly used as a quick file-stage utility. ⁴⁰
osmfilter	Straightforward tag filtering while keeping dependent objects. ⁴¹	Simpler semantics than a full ETL or SQL pipeline. ⁴¹	Shrinking OSM files to only roads/buildings/water/POIs before later processing. ⁴²	Useful for fast thematic preprocessing, especially before import into heavier systems. ⁴³

Example acquisition and preprocessing commands

For **targeted prototyping**, Overpass is a good way to get buildings, roads, water, and POIs into a notebook or editor tool. The query model, output formats, and bbox filters are documented in the Overpass manual. Public instances are fine for modest use; for large repetitive jobs, self-hosting is the right model. ⁴⁴

```
[out:json][timeout:60];
(
  way["highway"](37.770,-122.455,37.800,-122.400);
  way["building"](37.770,-122.455,37.800,-122.400);
  way["natural"="water"](37.770,-122.455,37.800,-122.400);
  nwr["amenity"](37.770,-122.455,37.800,-122.400);
);
out body;
>;
out skel qt;
```

When you are ready for a structured production store, `osm2pgsql` flex output is the recommended modern route, and `--slim` is required if you want to keep the database updateable later. ⁴⁵

```
osm2pgsql \
  --output=flex \
  --style=game.lua \
  --slim \
  --cache=4000 \
  --database osm_game \
  region.osm.pbf
```

For filtering, selection, and reprojection into an intermediate GIS file, `ogr2ogr` is the workhorse. ³⁷

```
ogr2ogr \
  -f GPKG roads_utm.gpkg roads.geojson \
  -t_srs EPSG:32610 \
  -select osm_id,highway,name,surface
```

Terrain, elevation, and georeferencing

Elevation source recommendations

OSM itself is not a DEM. For realistic terrain, roads draped to grade, river incision, and believable skyline/parallax, you need a raster or point-cloud elevation source. The right source depends almost entirely on geography and fidelity target. ⁴⁶

Elevation source	Coverage and characteristics	Strengths	Weaknesses	Best use
SRTM	Near-global coverage over about 80% of land; common products include 1 arc-second for the U.S. and 3 arc-seconds globally in the USGS archive. ⁴⁷	Free, stable, easy to automate, good baseline DEM. ⁴⁸	Too coarse for dense urban curb/embankment fidelity or close-up terrain realism.	Default global fallback when no better national DEM exists.
ASTER GDEM V3	Near-global land coverage from 83°N to 83°S at approximately 30 m spatial resolution. ⁴⁹	Broader land coverage than SRTM; useful fallback DEM. ⁵⁰	Still not a substitute for good LiDAR or national DEMs where precision matters.	Backup/global fill, especially where SRTM coverage is limited.
USGS 3DEP DEMs / LiDAR	U.S. high-quality topographic program; includes public-domain DEMs and lidar point clouds, including ~3 m 1/9 arc-second products in many areas. ⁵¹	Best practical source for accurate U.S. terrain; public domain; direct access via The National Map and LidarExplorer. ⁵²	U.S.-only.	Preferred source for any U.S.-based high-fidelity world.

A good rule is simple: use **3DEP/LiDAR first, national DEMs second, SRTM third**, and **ASTER as fallback or fill**. For gameplay, the most important derived products are: the gameplay heightfield, slope/roughness masks, water-surface flattening masks, road cut/fill adjustments, and optionally contours for debug and validation. ⁵³

DEM processing workflow

GDAL is the core raster toolbox here. `gdalwarp` handles reprojection and mosaicking; `gdal_translate` handles clipping and conversion; `gdal_contour` generates vector contour lines from DEMs. PDAL is the corresponding point-cloud layer when you start from LiDAR. ⁵⁴

A minimal DEM workflow is: clip to AOI, reproject into the local metric CRS, fill or mask no-data regions, resample to the terrain resolution your runtime can afford, then export both a render mesh/heightmap and a lower-cost collision copy. That separation matters because all three engines allow or encourage lower-cost collision representations than the full visual terrain. ⁵⁵

```
gdalwarp \
  -t_srs EPSG:32610 \
  -r cubic \
  raw_dem.tif dem_utm.tif
```

```
gdal_contour \
-i 5 \
dem_utm.tif contours_5m.gpkg
```

If you are starting from LiDAR, a realistic production workflow is **PDAL for point classification and gridding**, then **GDAL for final raster finishing and reprojection**. That split maps cleanly onto the strengths of the two libraries. ⁵⁶

Coordinate systems and georeferencing

OSM's native coordinates are latitude/longitude in WGS84, and GeoJSON under RFC 7946 also uses WGS84 geographic coordinates in decimal degrees. That is appropriate for interchange, but not for most game-engine measurements, mesh generation, or pathfinding costs. For those, you want a **local metric projected CRS** or a globe-aware Earth-centered frame with an explicit local tangent frame. ⁵⁷

The safest pattern is to carry **both** coordinate systems through the toolchain. Keep the original OSM object IDs and WGS84 coordinates so that you can resync with updates later, but perform all geometry-heavy build steps in a metric CRS such as UTM or a national projection. PROJ is the standard transformation layer; osm2pgsql flex can also write transformed geometries into tables with chosen projections. ⁵⁸

For engines, the georeferencing choice depends on world scale. In **Unity**, large-world precision problems stem from the single-precision world representation used by normal GameObjects, so you typically need either origin rebasing, Cesium's origin shifting, or Unity's high-precision framework. In **Unreal**, Large World Coordinates and World Partition are the native large-world answer. In **Godot**, large-world coordinates exist, but the docs are clear that you usually do not need them unless the world is truly huge. ⁵⁹

Feature translation, asset pipelines, and engine integrations

Buildings and roads

Buildings should not be imported as a single monolithic generic “extrude footprint” step. OSM supports `building=*`, `building:levels=*`, `height=*`, `building:part=*`, and the Simple 3D Buildings scheme, which means your translator should distinguish between plain footprints and structured multi-part buildings. If `height=*` is present, prefer it. If it is absent but `building:levels=*` exists, use that. If both are absent, use a local heuristic by district and building type. The OSM wiki explicitly notes that `building:levels` counts above-ground floors and that a common default rendering assumption where explicit height is absent is around 3 meters per floor. ⁶⁰

That leads to a robust building policy: import every building footprint; preserve OSM IDs and tags; compute a normalized height estimate; detect multi-part structures; then generate two meshes per chunk, one for visual rendering and one simplified collision proxy. Keep procedural roof and facade variation as a separate authored layer so you can restyle the world without invalidating the OSM-derived base geometry. ⁶¹

Roads should be translated from OSM geometry into an engine-specific combination of splines, ribbons, decals, or lane meshes rather than blindly triangulating linework. OSM ways carry ordered node sequences and relations encode route/turn structures; OSMnx is a useful bridge because it turns OSM into graph

structures you can simplify, measure, and validate. In gameplay terms, roads usually need four outputs: a visual ribbon/mesh, a drivable or walkable surface mask, a routing graph, and collision/nav source geometry. ⁶²

For accurate road alignment, roads need to be reconciled with the terrain rather than merely draped over it. Real-world roads often imply local grading, bridge decks, embankments, tunnels, and water crossings that a raw heightfield will not represent cleanly. That is one of the reasons a separate terrain-adjustment pass is important after vector import but before final mesh bake. OSM tags like `layer=*`, `level=*`, bridge/tunnel tags, and route/turn relations are signals you should keep through preprocessing even if your initial visual importer ignores them. ⁶³

A small but effective Python prototype layer looks like this with OSMnx. It is useful for road-graph iteration, routing experiments, and early asset-compilation tests before you lock the full production ETL. OSMnx's docs cover both graph and feature retrieval. ⁶⁴

```
import osmnx as ox

place = "Monaco"
G = ox.graph_from_place(place, network_type="drive")
G = ox.project_graph(G)

buildings = ox.features_from_place(place, tags={"building": True})
buildings = buildings.to_crs(G.graph["crs"])
```

Water, land use, vegetation, and POIs

OSM distinguishes **land use** from **land cover**, and that distinction is valuable for procedural world building. Land use encodes the human function of land; land cover describes the physical surface material. In OSM practice the distinction is imperfect and sometimes conflated, but from a game-pipeline perspective that is still powerful: land-use polygons can drive district logic and prop rules, while land-cover and natural tags can drive terrain materials, vegetation masks, and scattering density. ⁶⁵

For **waterways**, preserve both polygonal water bodies and linear waterways. Lakes and wide rivers often want polygonal meshes with flattened water surfaces; streams and narrow rivers often work better as spline-driven ribbons or flow meshes. Hydrology also feeds downstream gameplay systems such as shoreline placement, vegetation exclusions, floodplain dressing, and AI avoidance. OSM's relation model matters here because large water features and other complex polygons are often multipolygons rather than single closed ways. ⁶⁶

For **vegetation**, OSM is usually a mask, not a complete inventory. Use forest/wood/scrub/grassland-like areas and related land-cover signals as seed masks for procedural scattering, then modulate with DEM slope, height, and moisture proxies. This is one of the most effective places to add procedural augmentation, because the source semantics are strong enough to place the right biome in the right place, while the missing density and species detail can be filled cheaply and art-directably. ⁶⁷

For **POIs**, keep them lightweight. POIs are usually more useful as metadata than as heavy geometry: quest seeds, labels, commerce markers, fast-travel candidates, spawn hints, or destination nodes. Try not to turn every `amenity=*` or `shop=*` into a bespoke static mesh. OSM's open-ended tag system is a strength for metadata, but it also means your importer should normalize tags into a smaller gameplay-facing schema. ⁶⁸

Navmesh, collisions, and asset compilation

The recurring production mistake is to let the render mesh become the gameplay mesh. Do not do that. Compile at least three representations per chunk: a **visual mesh set**, a **collision representation**, and a **navigation representation**. On roads and terrain, those can sometimes share source geometry, but buildings, props, and detailed water edges should almost never use the visual mesh verbatim for physics and navigation. ⁶⁹

In **Unity**, the AI Navigation package provides high-level NavMesh components and supports building and using navmeshes at runtime and at edit time. Mesh colliders are accurate but more resource-intensive than primitive colliders, and the Terrain collider can use a lower-resolution `TerrainData` asset to save cost. ⁷⁰

In **Unreal**, use the engine's tile-based navmesh generation and, for very large maps, combine it with World Partition and the world-partitioned navigation mesh workflow. Landscape collision can be tuned via collision MIP levels, which reduces memory and speeds tests at the cost of accuracy. ⁷¹

In **Godot**, native procedural geometry and navigation systems are credible production tools. `ArrayMesh` gives you scriptable mesh construction, `NavigationMeshGenerator` handles 3D navmesh baking, large worlds can use chunked runtime navmesh baking, and `HeightMapShape3D` is faster than concave trimesh collision for terrain-like surfaces, though still slower than primitives. ⁷²

Engine integration and plugin options

Plugin ecosystems change quickly, so the table below focuses on the most durable, high-signal options rather than trying to catalog every community importer. The broad conclusion is that **Cesium is the mature choice for Unity/Unreal globe-scale georeferenced streaming**, while **custom offline preprocessing into engine-native assets** is the most robust choice for a conventional city/regional open world. ⁷³

Engine integration option	Pros	Cons	Typical use cases	Performance notes
Unity native pipeline + AI Navigation	Full control over asset schema; native terrain/nav/collision workflow. ⁷⁴	You must solve large-world precision and georeferencing yourself.	City/regional open worlds with offline-built chunks.	Favor additive scene streaming and simplified colliders; avoid excessive MeshColliders. ⁷⁵

Engine integration option	Pros	Cons	Typical use cases	Performance notes
Cesium for Unity	Quickstart includes Cesium World Terrain and Cesium OSM Buildings; supports origin shifting and georeferenced sub-scenes. ⁷⁶	Built around 3D Tiles / globe streaming rather than raw semantic OSM asset authoring.	Globe-scale apps, simulators, exact geodetic placement.	Strong for streamed content and precise georeferencing near the camera; less suited as the sole compiler for stylized offline worlds. ⁷⁷
Unity High-Precision Framework	Unity Technologies package for very large worlds addressing single-precision limitations. ⁷⁸	Not OSM-specific and not a complete geospatial pipeline.	Large local worlds where you want your own OSM/DEM compiler.	Helps with precision, but you still need chunking, streaming, and a GIS build pipeline. ⁷⁸
Cesium for Unreal	High-accuracy WGS84 globe, 3D Tiles, geospatial APIs, runtime streaming, and support for Unreal physics/collision/landscaping. ⁷⁹	Again, strong on streaming/georeferencing rather than raw OSM semantic compilation.	Digital twins, geospatial simulators, large exact-world experiences.	Pairs well with Unreal's large-world toolset and streaming systems. ⁸⁰
Unreal StreetMap plugin	Imports OSM XML into a StreetMap asset; sample component renders streets/buildings; roads keep connectivity data. ⁸¹	Community project, explicitly not officially supported by Epic; limited feature depth; mainly tested with direct OSM exports. ⁸¹	Fast prototype importer for roads/buildings in Unreal.	Sample renderer is intentionally simple and efficient, using a single draw call and quad-strip roads without tessellation or UV support. ⁸¹

Engine integration option	Pros	Cons	Typical use cases	Performance notes
Godot native procedural stack	Official docs support procedural meshes, chunked navmesh baking, large-world coordinates, and terrain-friendly collisions. ⁸²	More custom engineering than a drop-in importer.	Custom OSM pipelines, stylized or simulation-heavy worlds in Godot.	Bake nav chunks in the background; prefer <code>HeightMapShape3D</code> over concave terrain collision where possible. ⁸³
Terrain3D for Godot	High-performance clipmap terrain plugin with heightmap import, foliage instancing, and multiple LODs. ⁸⁴	Terrain system, not a semantic OSM importer.	Godot worlds that need serious terrain rendering after GIS preprocessing.	Supports large regions and multiple terrain/ foliage LODs; good fit after offline DEM compilation. ⁸⁴

Streaming, updates, licensing, validation, and implementation plan

LOD, streaming, and performance

For real-world worlds, **chunking and LOD policy matter more than almost any single import tool**. A practical chunk unit is a fixed metric grid or quadtree tile that contains: terrain, road ribbons, building meshes, land/water masks, collision proxies, nav source geometry, and a metadata table keyed by OSM object IDs. Compile near/mid/far asset representations separately so that you can stream small approximation sets instead of trying to keep the whole city in high detail. This is a design recommendation based on engine streaming behavior and OSM's large-area density patterns. ⁸⁵

In **Unreal**, World Partition is explicitly designed as automatic distance-based level streaming with grid cells, and it works with HLODs and world-partitioned navmesh. In **Unity**, additive scenes and multi-scene editing are the native building blocks for large streaming worlds. In **Godot**, chunked navigation mesh baking for large worlds is a first-class documented workflow. ⁸⁶

Storage and memory should be budgeted in layers. Keep raw source in PBF because it is compact. Strip unused tags before or during compilation. Use `osm2pgsql --flat-nodes` and bucket-indexed slim mode only when the world is large enough to justify the operational overhead. For planet-scale imports, `osm2pgsql` documents substantial savings from bucket indexes and flat nodes compared with storing all node locations in PostgreSQL tables. ⁸⁷

The same “compile separate representations” principle applies to collisions and nav. Unity’s docs recommend rougher colliders where possible and note that convex mesh colliders are more expensive than primitives. Godot’s docs make the analogous point by distinguishing primitive, convex, concave, and heightmap collisions. Unreal exposes collision-versus-accuracy trade-offs directly on Landscape. ⁸⁸

Update and patch workflows

If your shipped world is mostly static, the simplest workflow is still **versioned snapshots**: build from a dated regional extract, compile chunks, ship them, and do periodic full rebuilds. If your world is intended to track the living OSM database, then your pipeline has to be **replication-aware**. PBF headers can carry replication timestamp, sequence number, and base URL specifically so consumers can continue from the correct sync point. ⁸⁹

`osm2pgsql-replication` exists specifically to update an `osm2pgsql` database from an OSM replication server. It initializes from the import file or database state, stores replication status in the DB, then downloads and applies new changes. During updates it automatically uses append/slim semantics underneath. The `osm2pgsql` manual also recommends `PyOsmium` tooling for more custom update control. ⁹⁰

```
osm2pgsql-replication init \  
  -d osm_game \  
  --osm-file region.osm.pbf  
  
osm2pgsql-replication update \  
  -d osm_game \  
  -- \  
  --output=flex \  
  --style=game.lua
```

A few update caveats are important. Replication change files may contain multiple versions of the same object over the covered period, so simplified changes are often what you want for game-world patching. Change files are also not geometrically self-sufficient: changed ways may reference nodes that are not present in the diff, which is why persistent node-location caches or a synchronized snapshot/database matter. Finally, if you imported only a clipped extract but update from global planet diffs, your database can accumulate out-of-area objects over time unless you explicitly manage clipping during patch compilation. ⁹¹

The most production-friendly patch model is **tile-local rebuilds**. Map changed OSM object IDs to your compiled chunks, invalidate only touched chunks, rebuild those chunks’ terrain adjustments / roads / building meshes / nav / collision proxies, then publish only the changed asset bundles. That design also helps with attribution and QA because each shipped chunk can carry a source timestamp and extract lineage. This chunk-local invalidation strategy is a workflow recommendation based on the structure of OSM diffs and engine streaming systems. ⁹²

Licensing and legal considerations

OSM data is distributed under the **Open Database License 1.0**, which allows sharing, modifying, and using the database while preserving the same freedom for others. The OSM Foundation's attribution guidance says attribution must be visible to anyone exposed to the produced work, placed near the produced work or where users customarily expect it, and legible/understandable. ⁹³

The major legal distinction for game projects is between a **Produced Work** and an **adapted or derivative database**. OSMF guidance specifically notes that a map or other produced work can often be distributed under a license of your choosing, while an adapted database may need to be share-alike under ODbL if you distribute it publicly. That line depends on exactly what you distribute and how reconstructible the underlying database is. ⁹⁴

In practical engineering terms, maintain clear separation between: the original/adapted OSM-derived database, compiled game assets, and non-OSM procedural/art layers. That separation is not a substitute for legal analysis, but it makes attribution placement, source tracking, updates, and database-sharing questions much easier to manage. Also remember that **elevation data has its own licensing context**: USGS 3DEP data is public domain, and NASA EOSDIS pages note that SRTM products are openly shared without restriction; ASTER GDEM is available at no charge from LP DAAC/Earthdata. ⁹⁵

A practical attribution pattern for shipped games is an in-world or menu-visible credit such as “Map data © OpenStreetMap contributors,” placed somewhere the player can reasonably find without friction, alongside the citations/credits for DEM or LiDAR sources used. The exact wording and placement should be reviewed against the current OSMF attribution guidance and your distribution context. ⁹⁶

Testing and validation for geographic accuracy

Geographic accuracy should be validated in at least four layers: **geometry validity**, **topology/connectivity**, **coordinate transformation accuracy**, and **visual/metric correspondence to source data**. Geometry validity is straightforward with PostGIS or GDAL: `ST_IsValid`, `ST_IsValidReason`, `ST_MakeValid`, and `gdal vector make-valid` exist precisely for this job. ⁹⁷

A minimal database QA query for generated building footprints or water polygons looks like this: ⁹⁸

```
SELECT
  COUNT(*) AS total,
  COUNT(*) FILTER (WHERE NOT ST_IsValid(geom)) AS invalid_geom
FROM buildings;
```

For roads, validate both the geometry and the graph. OSMnx exposes street length, self-loop proportion, simplification, intersection counts, and related topology metrics, which are useful regression signals when you modify your translator. If a preprocessing change suddenly drops total road length or creates a spike in self-loops, you have likely broken the import. ⁹⁹

Coordinate QA should include **round-trip reprojection checks** and **spot checks against trusted control locations**. PROJ's model is explicit that transformations are sequences of operations between CRSs; in a

game pipeline, that means you should test forward and inverse transforms for representative points in every chunk and record the acceptable error budget. In addition, compare DEM-derived elevations against a small set of known checkpoints when vertical accuracy matters for gameplay. ¹⁰⁰

Finally, do visual QA with intent. The best practice is side-by-side validation of source vectors, source DEM, and final runtime assets at representative neighborhoods: dense downtown, suburban blocks, bridges/ramps, coastlines, water bodies, and steep terrain. Real-world accuracy issues tend to cluster at topology and vertical transitions rather than on simple flat residential blocks. That is an engineering inference based on the structure of OSM ways/relations and DEM limitations. ¹⁰¹

Sample implementation checklist and estimated effort

The ranges below are **engineering estimates**, not vendor estimates. They assume the target platform is unspecified, the AOI is not yet fixed, and the team is building a reusable pipeline rather than a one-off demo. The biggest drivers are AOI size, required terrain fidelity, amount of procedural dressing, and whether live OSM updates are required.

Task	Typical effort	Notes
Define AOI, fidelity targets, scale policy, and platform budgets	1–3 days	Decide early whether the world is local/projected or globe-aware.
Acquire baseline OSM + DEM/LiDAR sources	1–3 days	Faster for Geofabrik + SRTM; slower if national LiDAR acquisition is fragmented.
Stand up preprocessing stack	2–5 days	Install/configure osmium, osm2pgsql, GDAL/OGR, PROJ, Python tooling, DB.
Build initial schema and import pipeline	3–7 days	Includes PBF ingest, tagging normalization, CRS strategy, metadata retention.
Terrain processing and road/terrain reconciliation	1–2 weeks	Longer if bridges/tunnels and water flattening are important.
Building translator with height heuristics and collision proxies	1–2 weeks	Complex urban areas with many <code>building:part</code> relations take longer.
Road translator, routing graph, nav source geometry	1–3 weeks	Longer if traffic rules, lane logic, or pedestrian graphs matter.
Chunking, HLOD, and runtime streaming integration	2–6 weeks	One of the largest effort multipliers.
Procedural vegetation/props/missing-detail layer	2–8 weeks	Highly art-style dependent.
QA automation, regression metrics, and geometry validation	3–10 days	Much cheaper to automate early than to retrofit later.

Task	Typical effort	Notes
Replication/diff-based patching	1–3 weeks	Skip entirely if snapshot rebuilds are sufficient.
Licensing review, attribution UX, source provenance	2–5 days	Should happen before external distribution.

A realistic **technical prototype** for one city-sized AOI is often achievable in **3–8 weeks** by one experienced engineer with GIS comfort. A more production-ready pipeline with chunking, procedural augmentation, validation, and patching is more realistically **2–6 months** for a small multidisciplinary team. Those ranges widen quickly if you need globe-scale georeferencing, multiple engines, or high-fidelity LiDAR-derived urban terrain.

Open questions and limitations

Because the request left several parameters unspecified, a few recommendations intentionally stay broad:

- The report does **not** assume a specific AOI, so it does not inventory country-specific DEM/LiDAR sources outside the U.S.
- The report does **not** assume a runtime network connection. If live streaming is acceptable, Cesium-based options become relatively more attractive.
- The report does **not** assume an art style. A stylized world can tolerate more semantic correctness with less geometric fidelity; a simulation/digital-twin target usually needs more LiDAR, stricter CRS discipline, and more legal scrutiny around data provenance.
- The report does **not** assume a specific license strategy for your shipped data products. If you plan to distribute any OSM-derived database or editor-facing geodata, you should obtain project-specific legal review against the ODbL and current OSMF guidance.

¹ https://wiki.openstreetmap.org/wiki/Map_features

https://wiki.openstreetmap.org/wiki/Map_features

² ⁷ ¹⁴ ¹⁷ https://wiki.openstreetmap.org/wiki/OSM_file_formats

https://wiki.openstreetmap.org/wiki/OSM_file_formats

³ ¹⁰ ¹³ https://wiki.openstreetmap.org/wiki/Downloading_data

https://wiki.openstreetmap.org/wiki/Downloading_data

⁴ ⁵¹ ⁵³ <https://www.usgs.gov/3d-elevation-program>

<https://www.usgs.gov/3d-elevation-program>

⁵ ⁵⁹ ⁷⁷ <https://cesium.com/learn/unity/unity-placing-objects/>

<https://cesium.com/learn/unity/unity-placing-objects/>

⁶ ⁹⁶ https://osmfoundation.org/wiki/Licence/Attribution_Guidelines

https://osmfoundation.org/wiki/Licence/Attribution_Guidelines

⁸ ¹⁸ ⁸⁷ https://wiki.openstreetmap.org/wiki/PBF_Format

https://wiki.openstreetmap.org/wiki/PBF_Format

9 11 <https://www.geofabrik.de/data/download.html>
<https://www.geofabrik.de/data/download.html>

12 <https://dev.overpass-api.de/overpass-doc/en/targets/formats.html>
<https://dev.overpass-api.de/overpass-doc/en/targets/formats.html>

15 16 https://wiki.openstreetmap.org/wiki/OSM_XML
https://wiki.openstreetmap.org/wiki/OSM_XML

19 <https://wiki.openstreetmap.org/wiki/Planet.osm>
<https://wiki.openstreetmap.org/wiki/Planet.osm>

20 21 22 57 <https://datatracker.ietf.org/doc/html/rfc7946>
<https://datatracker.ietf.org/doc/html/rfc7946>

23 <https://gdal.org/en/stable/drivers/vector/shapefile.html>
<https://gdal.org/en/stable/drivers/vector/shapefile.html>

24 29 31 <https://docs.osmcode.org/osmium/latest/osmium-extract.html>
<https://docs.osmcode.org/osmium/latest/osmium-extract.html>

25 26 27 28 45 <https://osm2pgsql.org/doc/manual.html>
<https://osm2pgsql.org/doc/manual.html>

30 <https://docs.osmcode.org/osmium/latest/osmium-apply-changes.html>
<https://docs.osmcode.org/osmium/latest/osmium-apply-changes.html>

32 33 64 <https://docs.osmcode.org/pyosmium/latest/>
<https://docs.osmcode.org/pyosmium/latest/>

34 Release 2.1.0 Geoff Boeing
https://osmnx.readthedocs.io/_/downloads/en/latest/pdf/?utm_source=chatgpt.com

35 <https://osmnx.readthedocs.io/en/stable/internals-reference.html>
<https://osmnx.readthedocs.io/en/stable/internals-reference.html>

36 37 38 <https://gdal.org/en/stable/programs/ogr2ogr.html>
<https://gdal.org/en/stable/programs/ogr2ogr.html>

39 40 <https://wiki.openstreetmap.org/wiki/Osmconvert>
<https://wiki.openstreetmap.org/wiki/Osmconvert>

41 42 <https://wiki.openstreetmap.org/wiki/Osmfilter>
<https://wiki.openstreetmap.org/wiki/Osmfilter>

43 https://wiki.openstreetmap.org/wiki/Osmfilter_version_0
https://wiki.openstreetmap.org/wiki/Osmfilter_version_0

44 <https://dev.overpass-api.de/overpass-doc/en/>
<https://dev.overpass-api.de/overpass-doc/en/>

46 47 <https://www.earthdata.nasa.gov/data/instruments/srtm>
<https://www.earthdata.nasa.gov/data/instruments/srtm>

48 <https://www.earthdata.nasa.gov/data/catalog/lpcloud-srtmgl1-003>
<https://www.earthdata.nasa.gov/data/catalog/lpcloud-srtmgl1-003>

49 50 <https://www.earthdata.nasa.gov/data/catalog/lpcloud-astgtm-003>
<https://www.earthdata.nasa.gov/data/catalog/lpcloud-astgtm-003>

52 95 <https://www.usgs.gov/faqs/what-are-terms-uselicensing-map-services-and-data-national-map>
<https://www.usgs.gov/faqs/what-are-terms-uselicensing-map-services-and-data-national-map>

54 55 <https://gdal.org/en/stable/programs/gdalwarp.html>
<https://gdal.org/en/stable/programs/gdalwarp.html>

56 <https://pdal.org/>
<https://pdal.org/>

58 100 <https://proj.org/>
<https://proj.org/>

60 <https://wiki.openstreetmap.org/wiki/Key%3Abuilding%3Alevels>
<https://wiki.openstreetmap.org/wiki/Key%3Abuilding%3Alevels>

61 https://wiki.openstreetmap.org/wiki/Simple_3D_Buildings
https://wiki.openstreetmap.org/wiki/Simple_3D_Buildings

62 66 68 101 <https://wiki.openstreetmap.org/wiki/Elements>
<https://wiki.openstreetmap.org/wiki/Elements>

63 <https://wiki.openstreetmap.org/wiki/Node>
<https://wiki.openstreetmap.org/wiki/Node>

65 67 https://wiki.openstreetmap.org/wiki/Land_use
https://wiki.openstreetmap.org/wiki/Land_use

69 <https://docs.unity3d.com/6000.1/Documentation/Manual/mesh-colliders-introduction.html>
<https://docs.unity3d.com/6000.1/Documentation/Manual/mesh-colliders-introduction.html>

70 74 <https://docs.unity3d.com/6000.1/Documentation/Manual/com.unity.ai.navigation.html>
<https://docs.unity3d.com/6000.1/Documentation/Manual/com.unity.ai.navigation.html>

71 <https://dev.epicgames.com/documentation/unreal-engine/overview-of-how-to-modify-the-navigation-mesh-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/overview-of-how-to-modify-the-navigation-mesh-in-unreal-engine>

72 82 https://docs.godotengine.org/en/4.4/tutorials/3d/procedural_geometry/arraymesh.html
https://docs.godotengine.org/en/4.4/tutorials/3d/procedural_geometry/arraymesh.html

73 79 80 <https://github.com/CesiumGS/cesium-unreal>
<https://github.com/CesiumGS/cesium-unreal>

75 <https://docs.unity3d.com/ScriptReference/SceneManagement.LoadSceneMode.Additive.html>
<https://docs.unity3d.com/ScriptReference/SceneManagement.LoadSceneMode.Additive.html>

76 <https://cesium.com/learn/unity/unity-quickstart/>
<https://cesium.com/learn/unity/unity-quickstart/>

78 <https://github.com/Unity-Technologies/com.unity.gis.high-precision-framework>
<https://github.com/Unity-Technologies/com.unity.gis.high-precision-framework>

81 <https://github.com/ue4plugins/streetmap>
<https://github.com/ue4plugins/streetmap>

83 https://docs.godotengine.org/en/latest/tutorials/navigation/navigation_using_navigationmeshes.html
https://docs.godotengine.org/en/latest/tutorials/navigation/navigation_using_navigationmeshes.html

84 <https://godotengine.org/asset-library/asset/3892>
<https://godotengine.org/asset-library/asset/3892>

85 86 <https://dev.epicgames.com/documentation/unreal-engine/world-partition-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/world-partition-in-unreal-engine>

88 <https://docs.unity3d.com/2022.3/Documentation/Manual/physics-optimization-cpu-collider-types.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/physics-optimization-cpu-collider-types.html>

89 91 https://docs.osmcode.org/pyosmium/latest/user_manual/08-Working-With-Change-Files/
https://docs.osmcode.org/pyosmium/latest/user_manual/08-Working-With-Change-Files/

90 92 <https://osm2pgsql.org/doc/man/osm2pgsql-replication-1.9.1.html>
<https://osm2pgsql.org/doc/man/osm2pgsql-replication-1.9.1.html>

93 <https://opendatacommons.org/licenses/odbl/1-0/>
<https://opendatacommons.org/licenses/odbl/1-0/>

94 https://osmfoundation.org/wiki/Licence/Community_Guidelines/Produced_Work_-_Guideline
https://osmfoundation.org/wiki/Licence/Community_Guidelines/Produced_Work_-_Guideline

97 98 https://postgis.net/docs/ST_IsValid.html
https://postgis.net/docs/ST_IsValid.html

99 <https://osmnx.readthedocs.io/en/stable/user-reference.html>
<https://osmnx.readthedocs.io/en/stable/user-reference.html>